

DATABASE PROJECT PROGRESS REPORT

Library Management System - Phase 1 Conceptual Design

Project Information

- **Course:** Database Management Systems
- **Topic:** Library Management System
- **Stage:** Phase 1 Conceptual Design
- **Submission:** Progress report and ERD model
- **Group:** G5
- **Members:** Đinh Thế Phong, Nguyễn Hoàng Phúc, Nguyễn Gia Phát

I. Problem Context and Scope

1.1 Problem Statement

Modern library management faces a major challenge in integrating traditional and digital resources. Manual management of physical books leads to errors in tracking copy status, managing loan periods, and calculating overdue fines. Meanwhile, the rise of digital journals requires a complex access licensing and user authorization mechanism across students, faculty, and external guests. The system needs to resolve this inconsistency to optimize user experience and librarian management efficiency.

1.2 Objectives

- Build a centralized database to manage the entire catalog of resources (physical and digital).
- Track the detailed status of each physical book copy (Physical Copies) at different shelf locations.
- Automate the borrow/return process and the fine calculation system based on user classification.
- Manage digital access rights through license packages and access logs.
- Clearly distinguish roles and privileges among user groups: Students, Faculty, External Guests, and Authors.

1.3 Users and Main Workflow

- **Librarian:** Manage the resource catalog, approve borrow/return transactions, manage digital license packages, and process fine payments.
- **University Member:** Includes students and faculty. They have the right to search resources, borrow physical books to take home, and access licensed digital journals.
- **External Guest:** Only have the right to search and read physical materials on-site, or access limited public digital materials.

- **Author (Internal/External):** Creators or contributors of the resources (physical books or digital journals) provided to the library. They can be internal university members publishing research, or independent external experts.

Main Workflow: User searches for a resource -> Checks availability/License -> Executes a borrow transaction (for physical books) or logs access (for digital materials) -> Records return/Renewal -> Processes violations (if any).

1.4 Scope and Assumptions

- **In Scope:** Resource catalog management, user management, physical copy tracking, borrow/return workflow, fine management, digital access licenses, and system logs.
- **Out of Scope:** Library staff payroll management, infrastructure management (electricity, water, building maintenance), and external supplier procurement systems.
- **Assumptions:** Each university member has a unique identifier, and each physical book copy has a unique barcode for identification.

II. Functional Requirements

- **FR01 (User Management):** The system must store identification information for both internal members and external guests, including contact details and object classification.
- **FR02 (Resource Management):** The system allows classifying resources into physical books and digital journals with specific attributes (ISBN, ISSN, publication year).
- **FR03 (Copy Tracking):** Manage the status (available, borrowed, damaged) of each specific copy of a book title.
- **FR04 (Borrow/Return Execution):** Accurately record the borrow date, due date, and actual return date of the user.
- **FR05 (Fine Management):** Automatically generate a fine record when a user returns a book late or damages a material.
- **FR06 (Payment Transaction):** Log the users' fine payment transactions.
- **FR07 (Digital License Management):** Manage the allocation of digital journal access packages to individual users or user groups.
- **FR08 (Access Logging):** Record all access activities to digital resources for statistical and copyright control purposes.

III. Business Rules and Constraints

- **BR01:** External Users are only allowed to read on-site and are not permitted to execute transactions to borrow physical books home.
- **BR02:** A fine must always be linked to a specific borrow transaction or violation behavior.
- **BR03:** In each borrow transaction, a user can only borrow one unique book copy.
- **BR04:** Access to a digital journal can only be executed if the user currently owns a valid Granted License.

- **BR05:** Faculty members have a longer borrowing period and a higher maximum number of borrowed books compared to students.
- **BR06:** A physical book title can have multiple different physical copies stored in various locations.
- **BR07:** A resource can be co-authored by multiple authors, and an author can have multiple works.
- **BR08:** Every payment transaction must be associated with a unique transaction code from the financial system.
- **BR09 (Identity Management Constraint):** All individuals interacting with the system must exist in the **PERSON** superclass with a unique identifier. The hierarchy of internal subjects is standardized via Single Table Inheritance in **UNIVERSITY_MEMBER**, grouping student and faculty attributes under a specific classification flag.
- **BR10 (Physical Copy Tracking Constraint):** Borrowing and returning operations must never be executed at the generic **PHYSICAL_BOOK** catalog level. Transactions must strictly target a specific **BOOK_COPY** using its unique barcode to ensure precise accountability for physical damage or loss.
- **BR11 (Financial Flexibility Constraint):** Fines do not require a single lump-sum payment. The system permits multiple payment installments (**PAYMENT_TRANSACTION**), tracking progress via specific payment statuses.
- **BR12 (License Entitlement Constraint):** Access rights to resources linked via **RESOURCE_LICENSE** are locked by default. Borrowing or downloading is solely approved by the SQL system if a query confirms the user holds a valid, unexpired **GRANTED_LICENSE**.
- **BR13 (Log Decoupling Constraint):** To optimize performance for high write-throughput, the **ACCESS_LOG** table is completely decoupled from Foreign Key constraints with Master tables. Integrity checks by the Database Engine are bypassed in favor of independent indexes.

IV. Entities, Attributes, Relationships and Definitions

4.1 System Definition

The library management system is an integrated solution aimed at digitizing business processes, connecting users with knowledge sources efficiently and transparently.

4.2 User / Actor Definitions

A user is any individual interacting with the system, including entities with identifiers to execute borrowing or access rights.

4.3 Entity Definitions

4.3.1 PERSON

- **Definition:** Represents every individual with a profile in the system, including internal members and external authors.
- **Entity type:** Strong (independent) entity / Supertype.
- **Identifier:** `person_id`
- **Relationship meaning:** A Person can be the parent of 0..1 `UNIVERSITY_MEMBER` or 0..1 `EXTERNAL_AUTHOR`. Can be granted 0..N `GRANTED_LICENSE` packages and generate 0..N `ACCESS_LOG` histories.

4.3.2 UNIVERSITY_MEMBER

- **Definition:** Represents an internal university member (Student or Faculty) with the right to borrow physical books.
- **Entity type:** Subtype depending on PERSON.
- **Identifier:** `person_id` (Primary Key and Foreign Key)
- **Relationship meaning:** Inherits all information from PERSON. Can perform 0..N `BORROW` transactions.

4.3.3 EXTERNAL_AUTHOR

- **Definition:** Represents an independent external author contributing materials to the library.
- **Entity type:** Subtype depending on PERSON.
- **Identifier:** `person_id` (Primary Key and Foreign Key)
- **Relationship meaning:** Inherits all information from PERSON.

4.3.4 RESOURCE

- **Definition:** Represents the master catalog of all material types (physical or digital) in the library.
- **Entity type:** Strong (independent) entity / Supertype.
- **Identifier:** `resource_id`
- **Relationship meaning:** Is the parent of 0..1 `PHYSICAL_BOOK` or 0..1 `DIGITAL_JOURNAL`. Has 0..N authors via the `AUTHOR_OF` mapping table and requires 0..N access rights via `RESOURCE_LICENSE`.

4.3.5 PHYSICAL_BOOK

- **Definition:** Represents a physical book title that can be held.
- **Entity type:** Subtype depending on RESOURCE.
- **Identifier:** `resource_id` (Primary Key and Foreign Key)
- **Relationship meaning:** Inherits information from RESOURCE. A physical book can own 1..N `BOOK_COPY` instances.

4.3.6 DIGITAL_JOURNAL

- **Definition:** Represents a digital resource, article, or e-journal.
- **Entity type:** Subtype depending on RESOURCE.

- **Identifier:** `resource_id` (Primary Key and Foreign Key)
- **Relationship meaning:** Inherits information from RESOURCE. Associated with 0..N `ACCESS_LOG` access histories.

4.3.7 BOOK_COPY

- **Definition:** Represents a specific physical copy of a book title.
- **Entity type:** Strong entity partially dependent on PHYSICAL_BOOK.
- **Identifier:** `copy_id`
- **Relationship meaning:** Each copy belongs to exactly 1 `PHYSICAL_BOOK`. Can appear in 0..N `BORROW` transactions across different timestamps.

4.3.8 LICENSE_PACKAGE

- **Definition:** Represents a license package providing access rights to specific digital resources.
- **Entity type:** Strong (independent) entity.
- **Identifier:** `license_id`
- **Relationship meaning:** A package can contain 1..N resources via `RESOURCE_LICENSE` and be granted to 0..N users via `GRANTED_LICENSE`.

4.3.9 AUTHOR_OF

- **Definition:** Associative table storing the author-resource relationship.
- **Entity type:** Associative entity.
- **Identifier:** `person_id` + `resource_id`
- **Relationship meaning:** Resolves the many-to-many (N:M) relationship between PERSON and RESOURCE.

4.3.10 RESOURCE_LICENSE

- **Definition:** Associative table defining which digital resources require which license packages for access.
- **Entity type:** Associative entity.
- **Identifier:** `resource_id` + `license_id`
- **Relationship meaning:** Resolves the many-to-many (N:M) relationship between RESOURCE and LICENSE_PACKAGE.

4.3.11 BORROW

- **Definition:** Represents a user's physical book borrow/return transaction.
- **Entity type:** Strong entity (Transaction).
- **Identifier:** `borrow_id`
- **Relationship meaning:** Each transaction links exactly 1 `UNIVERSITY_MEMBER` with 1 `BOOK_COPY`. This transaction can generate a maximum of 0..1 `FINE` records.

4.3.12 FINE

- **Definition:** Represents a financial penalty when a user violates regulations (late return, damaged book).
- **Entity type:** Weak entity depending on BORROW.
- **Identifier:** `fine_id`
- **Relationship meaning:** Belongs to exactly 1 `BORROW` transaction. Can be paid via 0..N installment `PAYMENT_TRANSACTIONS`.

4.3.13 PAYMENT_TRANSACTION

- **Definition:** Represents a user's fine payment transaction.
- **Entity type:** Weak entity depending on FINE.
- **Identifier:** `transaction_id`
- **Relationship meaning:** Strictly belongs to exactly 1 `FINE` penalty.

4.3.14 GRANTED_LICENSE

- **Definition:** Represents the valid access right granted to an individual for a material package.
- **Entity type:** Associative entity / Event entity.
- **Identifier:** `grant_id`
- **Relationship meaning:** Resolves the permission granting relationship between 1 `PERSON` and 1 `LICENSE_PACKAGE` with a specific timeframe.

4.3.15 ACCESS_LOG

- **Definition:** System log recording each time a user accesses/downloads a digital document.
- **Entity type:** Weak entity (Event) depending on PERSON and DIGITAL_JOURNAL.
- **Identifier:** `log_id`
- **Relationship meaning:** Associated with exactly 1 `PERSON` and 1 `DIGITAL_JOURNAL` resource.

4.4 Attribute Definitions and Data Dictionary

4.4.1 PERSON

- `person_id` is the unique identifier of an individual. It serves as the primary key of the PERSON entity. Its data type is INT, and it is required and unique.
- `ssn` stores the social security number/national ID. Data type is VARCHAR(20), used as an alternative identifier.
- `name` stores the full name of the individual. Data type is VARCHAR(150), required.
- `address`, `email`, `phone` store contact information. Data types are VARCHAR(200), VARCHAR(100), and VARCHAR(15), respectively.
- `person_type` is used to classify the subject (Student, Faculty, Author). Data type is ENUM and is required.

4.4.2 UNIVERSITY_MEMBER

- **person_id** is the foreign key referencing PERSON and also the primary key.
- **member_id** stores the internal member code (Student ID or Faculty ID). Data type is VARCHAR(20) and is unique.
- **department** stores the affiliated department or faculty name. Data type is VARCHAR(100).
- **member_type** stores the internal role (STUDENT or FACULTY). Data type is ENUM.
- **major** stores the academic major. Data type is VARCHAR(100) (Applicable only to students).
- **academic_rank** stores the academic rank/degree. Data type is VARCHAR(50) (Applicable only to faculty).

4.4.3 EXTERNAL_AUTHOR

- **person_id** is the foreign key referencing PERSON and also the primary key.
- **affiliation** stores the name of the author's external organization or workplace. Data type is VARCHAR(150).
- **biography** stores the biography or professional notes. Data type is TEXT.

4.4.4 RESOURCE

- **resource_id** is the unique identifier of the resource. It is the primary key of the RESOURCE entity. Data type is INT, required and unique.
- **title** stores the title of the document or book. Data type is VARCHAR(250), required.
- **publish_date** stores the publication date. Data type is DATE.
- **topic** stores the scientific topic or genre. Data type is VARCHAR(100).
- **resource_type** classifies the document (Physical Book or Digital Journal). Data type is ENUM.

4.4.5 PHYSICAL_BOOK

- **resource_id** is the foreign key referencing RESOURCE and also the primary key.
- **isbn** stores the International Standard Book Number. Data type is VARCHAR(20) and is globally unique for that book title.
- **shelf_location** stores the shelf coordinates or physical location to find the book in the inventory. Data type is VARCHAR(50).

4.4.6 DIGITAL_JOURNAL

- **resource_id** is the foreign key referencing RESOURCE and also the primary key.
- **doi** stores the Digital Object Identifier. Data type is VARCHAR(100) and is unique.
- **file_size** stores the file size. Data type is INT (in KB/MB).
- **format** stores the digital document format (PDF, EPUB, XML). Data type is VARCHAR(10).

4.4.7 BOOK_COPY

- **copy_id** is the unique identifier for a physical copy. It is the primary key with data type INT.
- **resource_id** references the PHYSICAL_BOOK title. Data type is INT and is a required foreign key.
- **barcode** stores the unique barcode string affixed to the spine of each actual book. Data type is VARCHAR(50).
- **condition_status** reflects the physical condition of the copy (Good, Missing Pages, Torn Cover). Data type is VARCHAR(50).
- **availability_status** reflects the inventory status (Available, Borrowed, Under Maintenance). Data type is VARCHAR(20).

4.4.8 LICENSE_PACKAGE

- **license_id** is the unique identifier of the license package. It is the primary key with data type INT.
- **license_name** stores the commercial name of the package. Data type is VARCHAR(150), required.
- **provider** stores the name of the service provider or publisher. Data type is VARCHAR(100).
- **description** stores detailed descriptions of the license package's access scope. Data type is TEXT.

4.4.9 BORROW

- **borrow_id** is the unique identifier of the borrow transaction. It is the primary key, data type INT.
- **member_person_id** is a foreign key referencing UNIVERSITY_MEMBER, identifying the borrower.
- **copy_id** is a foreign key referencing BOOK_COPY, identifying exactly which book is borrowed.
- **borrow_date** stores the start date of the borrow transaction. Data type is DATE.
- **due_date** stores the deadline by which the borrower must return the book. Data type is DATE.
- **return_date** stores the date the system records the book as actually returned. Data type is DATE and can be NULL if the book is not yet returned.

4.4.10 FINE

- **fine_id** is the unique identifier for the fine. Primary key, data type INT.
- **borrow_id** is a required foreign key, linking the fine to the borrow transaction that caused the issue.
- **total_fine_amount** stores the total penalty amount. Data type is DECIMAL(10,2).
- **payment_status** stores the current payment status (Unpaid, Partially Paid, Completed). Data type is ENUM.

- `due_date` stores the deadline to pay the fine. Data type is DATE.

4.4.11 PAYMENT_TRANSACTION

- `transaction_id` is the unique identifier of the payment instance. Primary key, data type INT.
- `fine_id` is a required foreign key referencing the corresponding fine.
- `amount_paid` stores the actual amount paid in this transaction. Data type is DECIMAL(10,2).
- `payment_date` stores the date the payment was made. Data type is DATETIME.
- `payment_method` stores the payment method (Cash, Bank Transfer, Credit Card). Data type is VARCHAR(50).

4.4.12 GRANTED_LICENSE

- `grant_id` is the unique identifier of the license grant instance. Primary key, data type INT.
- `person_id` is a foreign key referencing the user granted the access right.
- `license_id` is a foreign key referencing the granted license package.
- `valid_from` stores the date the access right becomes effective. Data type is DATE.
- `valid_to` stores the date the access right expires. Data type is DATE. If expired, the system will automatically disable the document reading request.

4.4.13 ACCESS_LOG

- `log_id` is the unique identifier of an access instance. Primary key, data type BIGINT due to rapid data volume growth.
- `person_id` is a foreign key identifying the user who made the click.
- `journal_resource_id` is a foreign key identifying the requested digital resource.
- `access_timestamp` stores the exact time down to the second of the download instance. Data type is DATETIME.
- `ip_address` stores the IP address of the requesting device. Data type is VARCHAR(45).
- `download_success` stores the status of whether the document download was successful or failed. Data type is BOOLEAN.

4.5 Relationship Definitions, Cardinality and Participation

- **RESOURCE - AUTHOR (M:N):** A resource can have multiple authors, and an author can write multiple resources.
- **PHYSICAL_BOOK - BOOK_COPY (1:N):** A book title can have multiple copies.
- **UNIVERSITY_MEMBER - BOOK_COPY (M:N via BORROW):** A member can borrow multiple copies over time, and a copy can be borrowed by multiple different members.

- **BORROW - FINE (1:1 or 1:N):** A borrow transaction can generate a maximum of one overdue fine record.
- **PERSON - LICENSE_PACKAGE (M:N via GRANTED_LICENSE):** A person can be granted rights to multiple license packages, and a license package can be granted to multiple people.
- **DIGITAL_JOURNAL - LICENSE_PACKAGE (M:N):** A digital journal can be included in multiple different product packages.

○

4.6 Key Relationships and Cardinalities

The system maintains the following structural relationships to ensure data integrity:

Relationship	Cardinality	Interpretation
PERSON - UNIVERSITY_MEMBER	1 : 0..1*	A person may or may not be a university member.
PERSON - EXTERNAL_AUTHOR	1 : 0..1*	A person may have an external author role.
UNIVERSITY_MEMBER - BORROW	1 : N	A member can execute multiple borrow transactions.
BORROW - BOOK_COPY	N : 1	Each borrow transaction identifies a specific physical copy.
BORROW - FINE	1 : 0..N*	A fine may arise from a borrow; the final quantity depends on the policy.
FINE - PAYMENT_TRANSACTION	1 : N*	A fine can have a payment history (multiple installment records).

RESOURCE - PHYSICAL_BOOK	1 : 0..1	A resource can be classified under the physical book branch.
PHYSICAL_BOOK - BOOK_COPY	1 : N	A physical book title can have multiple physical copies.
RESOURCE - DIGITAL_JOURNAL	1 : 0..1	A resource can be classified under the digital journal branch.
RESOURCE - RESOURCE_LICENSE	1 : N	A resource can be associated with multiple license assignments.
LICENSE_PACKAGE - RESOURCE_LICENSE	1 : N	A license package can be utilized across multiple resources.
PERSON - GRANTED_LICENSE	1 : N	A person can receive multiple license grants.
LICENSE_PACKAGE - GRANTED_LICENSE	1 : N	A package can be granted to multiple persons.
PERSON - AUTHOR_OF - RESOURCE	M : N	The complex author-resource relationship is resolved via the AUTHOR_OF table.
PERSON - ACCESS_LOG	1 : N	A person can generate multiple access records.

DIGITAL_JOURNAL - ACCESS_LOG	1 : N	A digital journal can have multiple associated access records.
---	-------	--

V. Validation Cases

Test Code	Scenario Description	Expected Result
Test 01	An individual with the "External Guest" role attempts a request to borrow a "Book Copy" home.	The system rejects the borrow transaction.
Test 02	A user opens a "Digital Journal" when the related "License Package" has expired.	The system blocks document access rights.
Test 03	A user returns a physical book with a return date later than the due_date.	The system automatically generates a FINE record.
Test 04	A user executes a payment for an outstanding fine on the system.	Creates a PAYMENT_TRANSACTION and updates the fine status.